

Semantic Knowledge Graphs for LLM-Driven Source Code Analysis

*Co-authored by Dinis Cruz and ChatGPT Deep Research***

Introduction

Software source code is not just text – it’s a living system of interdependent components with rich structure and behavior. Treating an entire codebase as plain text leaves much to be desired, especially as codebases grow in complexity. In contrast to natural language, **source code has a specific hierarchical structure (files, classes, methods) and explicit dependencies** (imports, calls, inheritance) ¹. Simply grepping through code or using isolated file-by-file analysis fails to capture the bigger picture. This is where **semantic knowledge graphs** offer a transformative advantage. By mapping code into a network of typed nodes (e.g. functions, classes, variables) and labeled relationships (e.g. *calls*, *defines*, *inherits*), we can navigate and query the codebase as an interconnected graph of knowledge rather than disparate text blobs.

An example code knowledge graph illustrating relationships between code elements (functions, variables, etc.), enabling complex queries and insights that linear text analysis cannot easily provide.

In this white paper, we explore how semantic knowledge graphs – especially using a “graphs-of-graphs” approach – can dramatically improve the analysis and understanding of large source code repositories. We discuss why traditional relational databases (e.g. Postgres) fall short for this purpose, particularly when it comes to evolving schemas recursively, layering multiple graphs, and preserving rich context over time. To address these challenges, we present an architecture influenced by the work of Dinis Cruz, notably the **MGraph-DB** memory-first graph database and the **Type_Safe** modeling approach, combined with a multi-stage LLM pipeline (dubbed the **LETS** pipeline) for extracting and linking knowledge. We describe how large language models (LLMs) can parse and annotate source code (via AST analysis or directly using the model’s understanding) to produce semantically enriched graph nodes. These nodes are organized into layered ontologies/taxonomies that capture not only technical structure (function calls, data flows, dependencies) but also business metadata and behavioral semantics.

Throughout the paper we will illustrate a full end-to-end pipeline – from ingesting source code, to building multi-layer graphs stored in simple object storage (like Amazon S3) instead of a conventional database. We will demonstrate how context-aware queries over this knowledge graph can enable powerful analyses such as vulnerability mapping, attack surface enumeration, and reasoning about code behavior. We also discuss future integration of dynamic runtime data and business context into the model. All concepts are grounded in prior publications and projects by Dinis Cruz (e.g. MyFeeds.ai’s semantic LLM pipeline, The Cyber Boardroom initiative, and the OWASP OSBot projects), and we link these ideas back to the **LETS** model that emphasizes LLM-driven structured outputs.

1. Semantic Knowledge Graphs for Code: Graph-of-Graphs Approach

In essence, a **knowledge graph** represents information as a network of entities (nodes) and relationships (edges). When applied to source code, a knowledge graph encodes the key program **entities** – such as modules, classes, methods, variables – and the **relationships between them** – such as “function A calls function B”, “class X inherits class Y”, or “module M defines class X”. The graph is “semantic” because nodes and edges carry meaning (types, labels, attributes) rather than being generic connectors. By converting code into a semantic graph, we obtain a **structured representation of the code’s key components and the links between them** ². In other words, we extract the *who/what/how* of the codebase (which functions do what, who calls whom, how data flows, etc.) and encode those facts into a machine-readable graph structure ². Each source file or component can yield its own subgraph (e.g. an abstract syntax tree of that file), and these subgraphs interconnect, forming a “**graph of graphs**” – a higher-level graph where each node might represent an entire subgraph of a file or module.

Graph-of-Graphs Definition: “*Graphs-of-graphs (GoG) extends traditional graph theory by structuring individual graphs as nodes within a larger, interconnected graph.*” ³ In our context, this means we can treat each unit of code (say each microservice, or each subsystem, or each version of the code) as a graph in itself, and then have a higher-level graph that links these units. For example, every source file’s AST is a self-contained graph of its statements; the codebase knowledge graph can contain each file-graph as a node (or linked under a file node), and edges between file-nodes represent import relations or cross-file function calls. This hierarchical graph-of-graphs approach brings modularity and manageability – we can analyze or update one subgraph (one file or module) independently, then seamlessly merge it into the global graph as a single node or component.

Such an approach is very powerful for large codebases. It introduces a form of **hierarchical abstraction** – allowing one to zoom in on a single file’s details or zoom out to see high-level system dependencies. It also naturally supports **multi-perspective analysis**: since each subgraph can represent a different facet (syntax tree, control-flow graph, data-flow graph, etc.), we can overlay multiple graphs for the same code. Indeed, using graph-of-graphs, “*the knowledge graph can be extended with other types of graphs used in static analysis, such as data flow or control flow graphs*”, and even dynamic runtime information ⁴. For instance, one layer of the knowledge graph might capture the AST structure, another layer could add a data-flow graph linking variable definitions to usages, and yet another layer could incorporate a control-flow or state machine graph for each function. These layers together form a rich semantic model of the program.

Crucially, representing code as a semantic graph (rather than just plain text or isolated parse trees) enables **graph queries** and analyses that mirror how a developer or security analyst thinks about the code. Instead of manually combing through files, we can query the graph: e.g. “find all functions that write to `customer_data` variable but are not validating input,” or “show the chain of function calls from any web request handler down to the database layer.” Traditional static analysis tools struggle with such holistic queries, but a well-constructed code knowledge graph handles them gracefully. In fact, researchers have found that storing code in a graph allows answering complex questions that would be infeasible with text search or even basic AST analysis. As one example, by turning a codebase into a Neo4j graph, queries like “How many functions are defined in file X?” or “Which files use the variable `local_zone`?” become trivial, and an LLM agent can generate Cypher queries to answer these ⁵ ⁶. The graph stores relationships *within* and *across* files, enabling questions that span the entire

codebase context. “It shows the power of [a] knowledge graph which stores the relationships between/within files and source code, allowing us to answer questions that a text-splitting-based approach cannot” ⁷ .

Finally, semantic code graphs are a natural fit for capturing **evolving knowledge**. As the code changes or as we incorporate new analysis (new node types, new relationships), the graph can evolve organically. Nodes and edges can be added without having to redesign a rigid schema – a stark contrast to a traditional database. This dynamic extensibility is especially important for including higher-level semantic context (for example, linking code to business concepts or known vulnerabilities). We can continuously enrich the graph-of-graphs: one day we might add a “threat model” subgraph connecting to certain code paths, another day we might attach a “performance hotspot” subgraph based on runtime profiling. The core graph structure remains, with additional layers augmenting it. This recursive, layered growth of the schema is something semantic graphs handle naturally, whereas conventional relational schemas would buckle under the constantly changing tables and relations.

2. Why Traditional Databases Fall Short

One might ask: why not simply use a relational database or a document store to manage code analysis data? The reason is that **traditional databases struggle to accommodate the recursive, graph-oriented nature of code knowledge, as well as the need for fluid schema evolution and layered context**. A source code knowledge graph contains deeply nested relationships (e.g. function A calls B, which calls C, which accesses variable X in class Y, etc.) – representing and querying such recursive relationships in SQL is cumbersome and inefficient. While SQL databases can model graphs in theory (through join tables or recursive CTE queries), in practice they impose a rigid schema and heavy query cost for what graph databases handle with ease.

Moreover, as we analyze code, **our understanding of the schema itself evolves**. We might start by storing basic entities like *Function*, *Class*, *Module* and relations like *CALLS* or *DECLARES*. But later we may need to introduce new node types (e.g. *API Endpoint* or *Database Table*) and new relationships (like a *uses database* edge from a function to a DB table node). In a relational DB, adding these means altering tables or creating new ones, migrating data, etc. – a slow and potentially disruptive process. In contrast, a graph or JSON-based store can accept new node/edge types on the fly, enabling **recursive schema evolution** where the data model can recursively enrich itself. Dinis Cruz emphasizes the importance of this flexibility: modern applications often need “*flexible and organic graph structures (like Semantic Knowledge Graphs)*” that can grow and change shape easily ⁸ . A fixed-schema SQL setup simply can't deliver that kind of agility.

Another limitation is **graph layering and persistent context**. As described, we want to layer multiple analyses (graphs of graphs) – for example, overlaying a data-flow graph atop the call graph, linking a vulnerability metadata graph into the code graph, etc. Implementing these layers in a relational model would lead to a proliferation of tables and complex JOIN logic to simulate multi-level relationships. It becomes very hard to maintain the *context* of an analysis step or to isolate a layer. We also want to preserve context *over time* – e.g., maintain historical snapshots or different versions of the graph as the codebase evolves, and allow queries that compare these versions. Relational databases are not designed to version entire interconnected datasets easily, whereas treating the knowledge graph as data stored in versionable files (JSON files, for instance) makes it feasible to diff and track changes (much like code itself is tracked).

Perhaps most importantly, **performance and scalability for graph workloads** is a major issue. SQL databases excel at set-based operations and aggregations, but iterative graph traversals (like exploring a call chain or dependency path) can become expensive. Native graph databases or in-memory graph

representations do this far more efficiently. Dinis Cruz encountered these issues firsthand – while building *The Cyber Boardroom* platform, he needed to analyze and query complex security knowledge structures in a serverless environment. The available graph databases were “either too heavy, too complex to deploy, or just didn’t fit” his requirements ⁹. Relational options were unsuitable because the data needed to be manipulated as graphs and merged from JSON outputs frequently. This led him to conclude that a new approach was needed – one that favored in-memory graph handling with the ability to persist to simple storage.

In summary, **traditional relational databases fall short in enabling:**

- *Recursive schema evolution*: the ability to introduce new entity types and relationships on the fly as our understanding of the code grows.
- *Graph layering*: the ability to maintain separate but connected layers of graphs (structure, flows, metadata) without complex schema refactoring.
- *Persistent contextual modeling*: maintaining rich context (including historical versions or external annotations) attached to code entities in a first-class way.
- *Performance on graph queries*: especially in a serverless or real-time analysis setting where spinning up heavy SQL engines or performing many JOINS would be impractical.

Modern graph use-cases demand a simpler, more flexible persistence model. This is why our architecture does **not** use a traditional SQL database as the primary store. Instead, as discussed next, we leverage a memory-first graph database that persists to the filesystem (e.g. cloud object storage) in a lightweight manner. As Cruz notes, the ideal solution needed to “*use the file system as its main data store*” and avoid the deployment complexity of heavy graph DB servers ⁹. Storing the knowledge graph as JSON files (or similar) in S3 not only sidesteps the need for a constantly running database instance (saving cost in serverless scenarios), but also makes the data **portable, diffable, and easily merged**. The graph becomes a set of artifacts that can be version-controlled just like source code. This approach, combined with an in-memory engine for fast operation, gives us the best of both worlds: the agility of schema-free JSON with the query power of graphs.

3. Architecture Overview: MGraph-DB, Type_Safe, and the LETS Pipeline

To meet the above requirements, we propose an architecture grounded in Dinis Cruz’s prior research and tooling. The key pillars are:

- **MGraph-DB** – a memory-first, file-based graph database optimized for GenAI and serverless use cases.
- **Type_Safe model** – a strongly-typed object modeling approach to ensure consistency and validity of graph data.
- **LETS pipeline** – a multi-phase LLM-driven process (Large Language model Enabled Transformation & Semantics pipeline) for extracting and constructing the knowledge graph in stages.

At a high level, the system ingests source code and produces JSON graph representations which are then loaded into MGraph-DB for querying and analysis. Let’s break down these components:

3.1 MGraph-DB: Memory-First Graph Database

MGraph-DB (also referenced as **MGraph-AI** in its Python package form) is a graph database developed by Cruz to specifically address the shortcomings of existing solutions in modern AI-powered workflows. Its design principles make it an ideal backbone for our code analysis knowledge graph:

- **In-Memory Operation with JSON Persistence:** MGraph keeps the working graph in memory for speed, but persists data as JSON to disk (or S3) for durability ¹⁰. This yields *“the performance of an in-memory database with the reliability of persistent storage.”* ¹⁰ In practical terms, one can spin up an MGraph instance (in a serverless function, for example), load the JSON graph of a codebase from S3, perform queries or modifications in-memory, and then save back the JSON. When idle, no database process or connection needs to be alive – zero cost when not in use ¹¹. This is perfectly aligned with cloud and serverless architectures.
- **Lightweight and Serverless-Friendly:** Traditional graph DBs require running servers or clusters. MGraph-DB was built to be usable in AWS Lambda or similar environments, meaning it has minimal dependencies and starts up quickly. It *“could be executed on serverless environments”* and avoided being *“too heavy or too complex to deploy”* ⁹. It uses simple file I/O for persistence (which in our case can be S3 or an object store path) and thus sidesteps the need for complex installation.
- **Type-Safe and Layered Architecture:** MGraph-DB provides a *“robust, type-safe implementation with a clean, layered architecture that prioritises maintainability and scalability.”* ¹² All graph operations are organized into clear layers and classes – for example, an **Data layer** for read/query operations, an **Edit layer** for modifications, a **Filter layer** for search capabilities, and a **Storage layer** for persistence ¹³. This layering (as illustrated in MGraph’s design) separates concerns: the schema definitions (types of nodes/edges) lie in one layer, the business logic or model behavior in another, and the action interfaces (read/edit/filter/store) at the top [56+看图]. Such separation makes it easier to extend and evolve. The type-safe aspect means that every node/edge can be validated against expected types, preventing accidental inconsistencies (for example, an edge defined to connect a *Function* to a *Variable* will enforce that only those types link on that relation).
- **Graph-Specific Optimizations:** Features like high-performance in-memory graph traversal, rich attribute support on nodes/edges, and built-in semantic web concepts (ontologies) support are included ¹⁴. Unlike a general document database, MGraph inherently understands nodes and edges, so queries like “find neighbors of this node” or “filter nodes by attribute X” are first-class, efficient operations. It’s also designed to handle **version control** of graph data – since data is in JSON, you can diff versions of the graph or even store multiple versions, enabling time-travel or comparison queries on the code graph (e.g., how did the function call graph change from last release to this release).

Cruz’s motivation for creating MGraph-DB underscores its suitability. He notes that while developing The Cyber Boardroom (a platform dealing with complex security knowledge), existing DBs didn’t fit the bill, so using *“the latest features from the OSBot-Utils package, namely the Type_Safe classes,”* he built a new GraphDB to support use cases like serverless execution, fast AI/LLM lookups (Graph-assisted RAG), flexible semantic graphs, JSON/cloud storage, and easy diffing of graph data ⁹ ¹¹. Those are exactly the needs of our source code analysis scenario. Essentially, **MGraph-DB allows us to treat the entire code knowledge graph as a JSON document that can be loaded, merged, saved at will – but while active, it behaves as a speedy in-memory graph database.** This hybrid approach obviates the rigidity of SQL and the bulk of a huge graph DB server.

3.2 Type_Safe Model for Strongly-Typed Graph Data

To ensure reliability in an ever-evolving knowledge graph, we employ the **Type_Safe** modeling approach (originating from the OSBot utilities project by Cruz). Type_Safe is a Python-based mechanism to define data models with strict type validation. In essence, it lets us define classes or schemas for our nodes and edges with specific fields and types, and it will enforce those at runtime. This is extremely useful for a code knowledge graph: for example, we can define a schema that a node of type `Function` must have properties like `name: str`, `file: str (filename)`, `start_line: int`, etc., and relationships like `CALLS` should only connect a `Function` node to another `Function` node. The Type_Safe framework will raise errors or reject data that doesn't conform, preventing corruption of our semantic graph.

Cruz has stated that he uses the Type_Safe class "in just about everything I write" and in fact *"used [it] to create the MGraph_AI GraphDB"* ¹⁵. The documentation of Type_Safe was even automatically generated by an LLM (Claude) reading its source code, demonstrating how accurate and expressive the model is ¹⁵. In our architecture, Type_Safe definitions will underpin the ontology of the knowledge graph: we will formally define types like `Module`, `Class`, `Function`, `Variable`, `Parameter`, etc., along with allowed relationships (edges) such as `CONTAINS` (module → class or module → function), `CALLS` (function → function), `READS` (function → variable), `WRITES` (function → variable), `INHERITS` (class → class), and so on. Because of Type_Safe, if an LLM or any process tries to create an invalid edge (say linking a Variable to a Class with a relationship that's not allowed), the system will flag it. This gives us **the benefits of a schema (consistency, quality control)** without sacrificing flexibility – we can extend the schema with new classes as needed, and those too will be governed by Type_Safe rules.

In short, **Type_Safe turns the knowledge graph into a self-validating data structure**, catching mistakes early. It's like having the strictness of a compile-time type system, but applied to our graph data. This is particularly important when multiple automated stages (LLMs, scripts) are producing data – we want to trust but verify their output.

3.3 The LETS Pipeline: LLM-Driven Multi-Stage Processing

LETS – which we call the **LLM-Enabled Transformation and Semantics** pipeline – is a methodology to construct the knowledge graph in controlled, recursive stages using large language models. Rather than throwing raw code at an LLM and asking for a monolithic analysis (which would be a "black box" with no traceability), we break down the process into **modular steps, each producing structured output**. This approach draws heavily from Dinis Cruz's *MyFeeds.ai* project, where a similar multi-phase pipeline was used to analyze news articles and personas with deterministic, explainable results ¹⁶. As Cruz notes, splitting the task into discrete LLM calls with JSON outputs makes the workflow *"controlled and interpretable rather than one big black box."* ¹⁷ ¹⁸ Each stage has a well-defined input and output schema, enabling us to verify and preserve intermediate results (an essential aspect for debugging and trust).

For source code analysis, the LETS pipeline can be defined with stages analogous to those in MyFeeds (which had *Entity Extraction* → *Persona Graph* → *Relevance Mapping* → *Summary* ¹⁹ ²⁰). An example **pipeline for code** might be:

1. **Code Entity & Relationship Extraction (AST Parsing & Annotation):** *Input:* Source code file (or snippet). *Task:* Identify key entities in the code (functions, classes, variables, imports) and their relationships (function calls, class definitions, etc.), possibly with basic docstring or comment summaries. *Output:* A JSON structure (graph fragment) representing the code's AST and basic

relationships. Essentially, this is converting code into an initial graph form. We use either a traditional parser or the LLM itself to get an AST, then the LLM annotates that AST with semantic info. For example, for each function it might provide a short description, categorize it (e.g. “API handler” vs “utility”), and list what it calls. **This JSON is ingested into MGraph-DB as nodes/edges.**

2. **Contextual Graph Construction:** *Input:* Some contextual information relevant to analysis. This could be a **security policy graph** (e.g. an ontology of security requirements), a **persona or role context** (similar to MyFeeds personas, here it could be a “Security Auditor” context that highlights what is important – e.g. critical sinks, auth functions), or a **business domain graph** (mapping high-level business functionalities to code modules). *Task:* Build a semantic graph of the context – essentially an ontology of what concepts matter for this analysis. *Output:* JSON graph of the context. For example, a security context graph might have nodes like “InputValidationRoutine” or “DatabaseQuery” or “CryptoOperation” that represent concepts, and these might be linked in a taxonomy (e.g. “CryptoOperation” is-a “SecurityCriticalFunction”). Alternatively, if focusing purely on code, this stage could be omitted, but it becomes powerful when you want to tailor the analysis (just as MyFeeds did for different reader personas ²¹). In a business context scenario, this graph might include nodes for “Accounting Module” or “Customer Data” which we later link to code handling those concepts.
3. **Mapping and Integration:** *Input:* The code graph from stage 1 and the context graph from stage 2 (if used). *Task:* Align or map the code to the context, identifying overlaps or relevant connections. *Output:* A JSON structure listing the links between code entities and context entities. For example, it might map a specific function to a “uses database” concept, or tag a function as related to “Customer Data” because it processes customer_info. In a security use-case, this stage could flag potential vulnerabilities: e.g. “Function X calls `exec` on user input → map it to concept ‘CommandInjectionRisk’.” Essentially, this stage is where the LLM can reason about the code graph and the higher-level concepts to produce semantic annotations (like risk levels, relevance scores, etc.). The output is stored either as additional edges in the graph (linking code nodes to context nodes) or as attributes on code nodes (like `risk_level="high"`).
4. **Query or Summary Generation:** *Input:* The enriched, multi-layer knowledge graph (now containing code and context and their interlinks). *Task:* Generate either human-readable insights (reports, summaries) or answer specific queries. *Output:* This could be a textual report (e.g. a security assessment summary: “These 5 functions comprise the critical authentication logic, here is how data flows between them...”), or it could be further graph queries results. In some cases, this stage might be interactive: an analyst asks a question in natural language, an LLM translates that into graph queries, and the answers are retrieved (as demonstrated with Cypher queries in the earlier example ⁵ ⁶). The key is that at this final stage, we leverage the structured knowledge we built to deliver insights that are grounded in data. Since all intermediate steps are available, these insights are **traceable** back to the source code facts.

Each stage’s structured outputs are saved – nothing is ephemeral. This brings huge advantages in **traceability and determinism**. As Cruz notes in MyFeeds, “every LLM stage outputs a structured JSON file rather than free-form text,” yielding a provenance trail that explains final results ²². If the system flags a piece of code as vulnerable, we can point to the chain of evidence in the graph: “this function was marked as handling `CustomerPassword` data and lacks a call to the `validatePassword()` check, hence a weakness was noted” – all of which can be read directly from the graph relationships. The JSON outputs also make the process more deterministic: the LLM is guided to fill specific fields (reducing variability) ²³. If something goes wrong (say the JSON is malformed or misses an expected field), the system

knows to raise an error or retry that step ²³. This is far more robust than a single-pass LLM answer about the code, which might change with a slight prompt variation and is hard to systematically verify.

In implementation, the LETS pipeline can be orchestrated by a workflow engine or simply by sequential function calls (possibly using tools like Prefect flows, which Cruz integrated into OSBot-Utils for orchestration ²⁴ ²⁵). The key is that each stage can be run independently and logged. In our architecture, we imagine kicking off the pipeline when, say, a new code repository is ingested or when a scheduled analysis is run. The LLM prompts at each stage are carefully designed with **schemas**. For example, in stage 1, the prompt might say: *“Here is a source file. Extract all functions, classes, imports, and output a JSON with lists of these entities and their relationships (function calls, etc.) in the following format...”* We may even include a brief ontology description so the LLM knows the types to use. Initially, we might “let the LLM pick the best relationships” on its own, without enforcing a strict ontology, and observe its output ²⁶. Cruz did this in the first iteration of MyFeeds (for entity extraction from text) and found the LLM did a decent job choosing meaningful relations ²⁶. Over time, we can incorporate a stricter ontology/taxonomy in the prompt to improve consistency, essentially moving from a more open-ended extraction to a controlled one as we refine the model ²⁶.

In essence, the LETS pipeline turns the codebase into data, then enriches that data step by step, similar to how a compiler might have multiple passes. But here, the “passes” involve intelligence from an LLM to imbue the raw code graph with higher-level semantics. Each pass feeds into the next, and the end result is a **multilayered knowledge graph ready for deep inquiry**.

4. LLM-Powered Source Code Analysis and Graph Generation

One of the most innovative aspects of this approach is the use of **Large Language Models to bridge the gap between raw source code and semantic graph representation**. Traditional static analysis might stop at constructing an AST or call graph; we go further by letting an LLM interpret and annotate the code, effectively turning unstructured (or semi-structured) code into a richly annotated knowledge graph.

4.1 Breaking Down Code with ASTs and LLMs

The process typically begins with parsing the source code into an Abstract Syntax Tree (AST) using a standard parser for the language. The AST provides the fundamental structural graph: e.g., a `Function` node with children `Parameter` nodes, or a `Class` node containing `Method` nodes, etc. As noted in an article by Dinis Cruz, once you have code as an AST, “a whole world of opportunities opens up” because the code is now a manipulable object graph ²⁷. We leverage this by taking the AST (which is effectively a graph of the code structure) and converting it to a JSON form that an LLM can process (or we can directly feed the code text, but structured input tends to yield better structured output).

Next, we prompt the LLM to enrich this AST-derived information. For example, for each function defined in the code, we might ask the LLM to provide: a brief description of its purpose, what broader category it falls into (e.g. “controller”, “utility”, “data-access”, etc.), what important external calls it makes, and any notable security or performance characteristics. The LLM can also identify relationships that are not explicit in the AST. For instance, an AST will show that function `foo()` calls function `bar()`, but the LLM could further annotate that relationship as “uses authentication routine” if `bar()` happens to be an auth function – something a pure AST wouldn’t label. In other words, the LLM adds **semantic annotations** on top of the syntactic structure.

To ensure consistency, we supply the LLM with a schema for the output JSON. Just as Cruz used LLM (Claude) to generate documentation by reading source code and outputting it in a structured format ¹⁵, we use a similar approach for analysis. The prompt clearly defines the JSON keys and expected types (leveraging the ontology from our Type_Safe model). An example output for a function might look like:

```
{
  "entity": "Function",
  "name": "processOrder",
  "in_file": "orders.py",
  "calls": ["validateOrder", "calculatePrice", "saveOrder"],
  "reads": ["user_input"],
  "writes": ["order_record"],
  "doc": "Processes a customer order by validating input, computing price,
and saving to DB.",
  "tags": ["IO", "Database", "Critical"]
}
```

This is a simplified illustration, but it shows how an LLM could take the code of `processOrder` and emit a JSON node with relationships (calls, reads, writes) and semantic tags or a docstring summary. The **ability of modern LLMs to understand code in natural language terms is impressive** – as evidence, Claude was able to read the *Type_Safe* class’s source and produce accurate documentation of its behavior without external hints ¹⁵. Similarly, GPT-4 or others can read a function and describe what it does. We harness that capability to populate our graph with meaning.

One might wonder: why not use static analysis tools for this? Certainly, many relationships (calls, data flows) can be obtained via static analyzers. We do use them for the raw graph. But the LLM adds a layer of **human-like insight** on top. It can infer the intent of code (e.g. “this function is a factory method” or “this class implements a strategy pattern”), which is extremely hard for static tools to label. It can also standardize disparate information. For instance, different modules might have inconsistent or missing documentation – an LLM can generate descriptions for each, giving us uniform coverage of semantic info. In security analysis, an LLM can recognize a known insecure coding pattern even if it’s not explicitly labeled as a vulnerability by linters, by drawing on its training knowledge. Essentially, LLMs act as a smart assistant to tag and explain the code graph.

We do this iteratively for all parts of the codebase. Each source file’s analysis JSON is then converted into graph nodes and edges in our MGraph-DB. Conveniently, MGraph-DB was designed to *“easily manipulate and merge those JSON objects as nodes and edges”* ²⁸ – meaning we can take the LLM’s JSON output and feed it directly into MGraph, which will instantiate the corresponding graph elements. This smooth integration (JSON -> graph) is a cornerstone of our pipeline’s efficiency. Instead of writing complex translators, we lean on MGraph’s ability to ingest JSON-defined graphs. Cruz notes that MGraph-DB was a “critical part” of the MyFeeds workflow for exactly this reason ²⁸.

4.2 Semantic Annotations and Ontology Alignment

After the initial extraction, we proceed to higher-level semantic augmentation (stage 2 and 3 of LETS as described). Here, the LLM operates not on raw code, but on the graph data itself (which can be represented as JSON or a simplified tree for prompting). For example, we might prompt: *“Given the following function definitions (with their summaries and calls) and a list of security-critical API calls, identify which functions perform authentication.”* The LLM can then output something like: `{"function":`

`"loginUser", "annotate": {"auth_related": true, "uses": ["verifyPassword"]}]}`. We incorporate those annotations back into the graph (perhaps adding a tag or linking the node to an “Authentication” concept node). By iterating like this, the graph becomes richer.

We also start to enforce the ontology more strictly at this phase. If we have a predefined taxonomy of, say, data sensitivity levels or component types, we prompt the LLM to classify each code entity into that taxonomy. For example: *“Classify each function as one of {UI Controller, Business Logic, Data Access, Utility} based on its name and calls.”* The result could then attach a property `category="Data Access"` to certain functions that talk to the database. This aligns the free-form understanding of the LLM with our structured ontology. Initially, as mentioned, we might have let the LLM choose relationships freely; as we mature, we provide the exact allowed relations (ontology) to use ²⁶, yielding more deterministic graphs.

One fascinating outcome of using LLMs is that they can sometimes *infer relationships that are not explicitly in code but make logical sense*. For instance, consider two modules that don’t call each other but have similar naming or documentation – an LLM might infer they are related conceptually (e.g. both part of “payment processing”). We could allow the LLM to suggest adding a relationship like *SimilarTo* or *ConceptuallyRelated* between such modules. This becomes an additional “semantic layer” of the graph that purely code-based analysis would miss. It’s akin to how a developer reading code builds a mental map of concepts and grouping – the LLM can provide that and we can capture it in the graph.

After applying the LLM at various granularities (function-level, module-level, system-level summarization), we end up with a comprehensive semantic graph. Each node in the graph is richly annotated (with descriptions, tags, risk scores, etc.), and the graph has additional nodes that represent abstract concepts (like “DataPrivacy” or “PerformanceCritical”) which are linked to code entities that fall under those concepts.

Because all data is stored as JSON and loaded in MGraph, we **persist every bit of context we derive**. This persistent contextual modeling means that if we run the analysis today and then run it again after a code change, we can diff the JSON graphs to see not just code differences, but analysis differences (e.g., a function’s risk rating changed). This persistence is important for auditing and for incrementally updating the knowledge graph (we don’t need to recompute everything from scratch if only one module changed; we can update that part’s JSON and merge it).

4.3 Ensuring Quality and Handling LLM Limits

It’s worth noting that using LLMs for code analysis requires careful prompt engineering and validation. We utilize the Type_Safe system and JSON schema validation to check LLM outputs. If the LLM produces invalid JSON or something that doesn’t match the schema, we detect it and can either correct via a follow-up prompt or discard that part. This guards against “hallucinations” where an LLM might, for example, invent a function that doesn’t exist. Since we base the analysis on actual ASTs and code context given to the LLM, the risk is mitigated, but our pipeline has sanity-checks at each step (for instance, cross-validating that if LLM says function A calls B, that edge actually exists in the AST graph, otherwise ignore it).

Additionally, we manage the scope of what code the LLM sees at once to stay within token limits. By breaking files and using iterative prompting, we ensure we’re not overloading the model. In practice, analyzing one file at a time (or one module) is feasible with current models. The *graphs-of-graphs* design helps here: we process each subgraph in isolation with the LLM, rather than trying to feed a huge monolithic codebase at once.

The outcome of this LLM-driven analysis is that we have effectively **documented and indexed the codebase’s knowledge in a queryable form**. Even if the original code had sparse comments or was developed by many teams with inconsistent practices, our pipeline generates a consistent knowledge layer over it. Every function has a description, every important relationship is captured, and higher-level patterns are noted.

The power of this approach was echoed in MyFeeds (for text), where even using just titles and summaries of articles, the LLM-generated graph produced *“a dense web of interconnected entities, illustrating how much structured information can be derived from raw text.”* ²⁹. Analogously, from raw source code we can derive an incredibly rich graph of interconnections. Even large codebases can be shrunk to a concise knowledge graph that a CISO or architect could explore without reading thousands of lines of code. The heavy lifting of understanding is done by the combination of static parsing and LLM semantic augmentation.

5. Layered Ontology: Technical, Business, and Behavioral Metadata

Building an effective knowledge graph for code requires a well-thought-out **ontology and taxonomy** – essentially, a schema of types and relationships that span multiple concerns. We are not just interested in the technical structure of the code; we also want to incorporate business context (what domain or feature a piece of code relates to), and behavioral metadata (how the code behaves at runtime, how users or external systems interact with it, etc.). Organizing these facets in a layered model helps maintain clarity.

Let’s break down the layers of the ontology:

- **Technical Code Ontology (Core Layer):** This is the fundamental schema of code entities and their relations. It includes node types like *Repository*, *Module/Package*, *File*, *Class*, *Function/Method*, *Variable/Field*, *Constant*, etc. Relationships here are things like *CONTAINS* (e.g. Repository contains Module, Module contains File, File contains Class, Class contains Method), *CALLS* (Function → Function), *IMPORTS* (File/Module → Module), *INHERITS* (Class → Class for inheritance), *IMPLEMENTS* (Class → Interface, in languages with interfaces), *SETS/READS* (Function → Variable), *THROWS* (Function → Exception type), etc. This layer essentially captures the same information a combination of an AST and a dependency graph would provide. It’s the skeleton of the codebase. We rely on this as the base graph on which other layers attach. Because it’s so crucial, it’s rigorously type-safe (all these relationships are explicitly defined). The *schema hierarchy* might resemble known code ontologies; for instance, academic works often define ontologies with classes like *Function*, *Loop*, *Condition*, etc., but we will focus on the higher-level program structure classes to keep the graph tractable.
- **Security and Quality Ontology (Vulnerability/Risk Layer):** On top of the raw structure, we introduce concepts relevant to application security (AppSec) and code quality. This layer includes abstract nodes like *Vulnerability*, *Threat*, *SecurityControl*, *ComplianceRequirement*, *CodeSmell*, *PerformanceIssue*, etc. For example, a *Vulnerability* node might have subtypes (an ontology of CWE entries or custom risk categories: SQL Injection, XSS, HardcodedCredential, etc.). We link these to code nodes that are affected. If our LLM or analysis detects that a function `login()` is missing input sanitization, we could create a node *InputValidationMissing* (a type of vulnerability) and connect an edge `ISSUE_IN` → (Function `login`). Similarly, a *SecurityControl* node like *Encryption* might be linked to functions that perform encryption, or a *ComplianceRequirement* like *GDPR/PIIDataHandling* might link to modules that deal with personal data. This layer allows

context-aware queries such as “show me all high-risk vulnerabilities in code handling payment information” – which would traverse code → business layer (payment info) → vulnerability links.

- **Business Domain Ontology (Domain Layer):** This layer maps the code to business concepts and metadata. It might include nodes for *Feature*, *Product*, *BusinessCapability*, *Team/Owner*, *Microservice*, *InfrastructureComponent*, etc., depending on what context is available. For instance, if we know Module A is part of the “ShoppingCart” feature, we create a node *Feature:ShoppingCart* and link Module A to it (relationship *IMPLEMENTS_FEATURE* or similar). If Class X is particularly critical for an SLA (say it’s part of order processing which has a KPI), we can encode that as well. This layer answers the question “What does this code *mean* for the business?”. A CISO or founder reading the analysis might not care about a function `calcTax()` in isolation, but if we present it as “`calcTax()` in Module Billing, which is part of Feature Invoicing (owned by Finance Team)”, it carries a lot more context. Business nodes can also represent **personnel context** – e.g., *Team Alpha* node connected to the modules they maintain, or *Service XYZ* connected to the repository if the codebase is microservices. This way, queries like “which team’s code has the most security issues?” become possible by traversing from Vulnerability nodes to Code to Team.
- **Behavioral & Dynamic Metadata Ontology (Execution Layer):** While static code structure is central, understanding actual behavior often requires dynamic data. This layer is where we would integrate things like *runtime performance metrics*, *usage frequency*, *test coverage*, *dependency runtime versions*, etc. For example, a *RuntimeTrace* node could represent an observed execution path (maybe from a trace of a request in production) linking functions in the order they were called. Or a *Coverage* node might attach to a function indicating it’s covered by tests (or not). We might represent user roles or personas that exercise certain functionalities – similar to how MyFeeds had personas for content, one could have “Attacker persona” graph that highlights likely attack paths, or a “User journey” graph mapping steps a user takes in the application to the code handling those steps. The possibilities are broad. The point is to have a place in the ontology where this dynamic and behavioral info can live. Initially, we might not populate much of this (since it requires external data), but designing the ontology with this in mind “future-proofs” the system. Indeed, researchers suggest incorporating runtime data like test coverage can greatly enhance the analysis ⁴, and our architecture is ready to include it when available.

Each of these layers can be thought of as a subgraph or a facet of the overall knowledge graph. We maintain links between layers. For instance, a Function node (technical layer) can have an edge to a Vulnerability node (security layer) and another edge to a Feature node (business layer). Through these connections, one can traverse across layers to answer complex queries.

Example Ontology in Practice: Consider a specific code element – say the `CheckoutService.processPayment()` method in an e-commerce system. In the technical layer, this is a Function node with edges to the class `CheckoutService`, to the functions it calls (maybe `chargeCard()` etc.), and to variables it uses. In the business layer, we link this function’s module (or service) to a Feature node “OnlinePurchase” and perhaps to a Team node “PaymentsTeam”. In the security layer, our analysis might have identified that `processPayment()` logs sensitive data, so we create a *CodeSmell:LoggingPII* node and link it to `processPayment()` with an edge *HAS_ISSUE*. We also tag it as *Critical* because it deals with money, linking it to a Compliance node “PCI-DSS” (since payment processing must be PCI compliant). In the behavioral layer, we might attach a metric node indicating this method was called 1,000 times/day on average (from monitoring data).

Now imagine the kinds of questions we can answer: “Show me all functions in the `OnlinePurchase` feature that handle PII and how often they are executed.” This query would find `Feature:OnlinePurchase`, get linked code entities, filter those that have edges to a “PII” or compliance node, then bring in any execution frequency metrics. Answer: perhaps `processPayment()` and `saveCustomerInfo()` are such functions, called X times per day, and have issues Y and Z. For a CISO or risk officer, this directly connects technical risk to business impact.

Designing the ontology is an iterative process. We start with broad categories and refine as needed. Dinis Cruz’s strategy, as seen in MyFeeds, was first to let the AI output relationships freely, then gradually impose a human-defined schema to sharpen the determinism ²⁶. We anticipate similarly that as we run the pipeline, we’ll discover new relationship types or node classifications that are useful. We can then update our `Type_Safe` schema to include them and re-run or adjust prompts.

One challenge is mapping the LLM’s outputs to the ontology. We mitigate this by providing the LLM with the ontology vocabulary in the prompt. For example: “Use the following categories for function roles: {Controller, Service, Utility, ExternalAPI}”. This way, the LLM’s annotations will align with our nodes. If it strays (like inventing a category “Helper” which we didn’t define), we either map it to an existing category or extend the ontology if it makes sense.

To give a concrete reference: in the MyFeeds persona graph example, the LLM was not initially given a fixed ontology, yet it identified sensible relationships like “Angel Investor works_with Techstars, uses Network Security, manages Funding Rounds”. Later, one could enforce a stricter set (e.g., relationship types like *invests_in*, *concerned_with* etc. if formalizing an ontology of personas) ²⁶. In our case, early on the LLM might output some ad-hoc tags for code (like “performance-critical”), and if we see value, we formalize that into our ontology (perhaps as a boolean property or a link to a Performance concept). The system thus becomes smarter and more structured over time, blending the LLM’s implicit knowledge with our explicit ontology design.

6. End-to-End Pipeline and System Architecture

Having described the components, we now paint the complete picture of the system architecture – from source code ingestion all the way to multi-layer graph storage and querying. The architecture can be visualized as a flow of data through various stages and storages:

6.1 Source Ingestion: The process kicks off by pulling source code from a repository. This could be a GitHub repository, a local codebase, or even multiple repositories (monorepo or multi-repo scenario). In a continuous integration setup, this might be triggered by a new code push or a scheduled job. The code is fetched (and possibly checked out at a certain commit or tag for repeatability of analysis).

6.2 Parsing and Initial Graph Construction: Next, we run language-specific parsers to generate ASTs for each source file. We convert these ASTs into a baseline graph representation (either directly constructing an in-memory graph or serializing to an intermediate JSON). At this point, we also collect any existing documentation or metadata – for example, function docstrings, code comments marked with `TODO` or special tags (like `@deprecated` annotations) – and attach them as raw text notes in the intermediate structure. The output of this stage is a *low-level code graph* (largely syntax and dependency oriented). We might store this as JSON files, e.g., one JSON per file or per module, capturing the structure. These can be stored in S3 as `graph_raw/moduleX.json` for instance. Storing intermediate results is helpful for traceability and caching – if code hasn’t changed, we can reuse its AST JSON.

6.3 LLM Analysis Stages: Now the LETS pipeline proper takes over. A controller (which could be a simple Python script orchestrating or a Prefect/Dagster flow) will iterate through the files/modules and invoke the LLM for each, according to Stage 1 (entity extraction/annotation). For each input (be it a raw code string or an AST-derived summary), the LLM returns a JSON snippet as described. These snippets are written to storage (e.g., `graph_semantic/moduleX.json`). Immediately after each LLM call, we validate the JSON against the schema. If valid, we merge it into the growing knowledge graph; if not, we may log an error and attempt a reprompt or fix (possibly using some few-shot examples to correct common errors).

Once all code files have been processed with Stage 1, we have a first-pass **Semantic Code Graph** stored (this can be thought of as Graph Level 1). We load all these JSONs into MGraph-DB in memory to form a unified graph of the entire codebase. MGraph's merging capability means if, say, file A and file B both reference the same class C, and they produce separate JSON nodes for class C, when merged we unify them into one node (assuming we key by a unique ID or name). The Type_Safe definitions help in this merge – for instance, the class named `User` in module X should be one node, even if multiple files mention it; we ensure the merge by a consistent identifier (like `Class:User@X`). MGraph-DB's design for easy merging of JSON was a key reason it's used ²⁸.

Next, Stage 2 (context graph construction) is run if we have additional context. Let's say we have a predefined security ontology or we programmatically create some context nodes (like one representing each OWASP Top 10 category, or one for each high-level component defined in architecture docs). These could be input as simple YAML/JSON that we convert to graph form. Or we could use LLM to generate a context graph from a prompt like "list key assets and threat categories for this application domain." Either way, we obtain a **Context Graph** (Graph Level 2). This too is loaded into MGraph, but typically we keep it separate until linking.

Stage 3 (mapping/alignment) is then executed. We prompt the LLM with combinations of code graph data and context graph data. We might do this in many small steps or a few big steps, depending on complexity. For example, to map security issues, we could go function by function: "Given the following function info and a list of common vulnerability patterns, output any that apply to the function." Or we could go vulnerability by vulnerability: "List all functions (from this JSON list) that relate to \$VULN pattern." The LLM outputs mapping info, like a list of tuples (function → vuln) or (function → concept). Those are then turned into relationship edges in the MGraph (e.g., `Function processPayment ->linked_to-> PCI_Compliance`). After this stage, the previously separate context graph becomes connected to the code graph, forming one multi-layer graph.

Because MGraph-DB operations (Data, Edit, Filter, etc.) are available, some mapping can also be done by algorithmic means. Not everything needs an LLM. For instance, we could automatically link any function named `*Test` to a concept `UnitTest` or link any SQL query usage to concept `Database`. But for subtle connections, the LLM helps.

At the end of Stage 3, we now have **Graph Level 3: an enriched, multi-layer Knowledge Graph** representing code + context + discovered links. This is stored back to the object store (S3) as well – perhaps as one consolidated JSON (since MGraph can serialize the whole graph to JSON). We version this file (or set of files) – e.g., `analysis_run_2025-05-30.json` – which can be several MBs to 100s of MBs depending on codebase size. (The storage is cheap and the JSON can be gzipped; since this is for analysis, we prefer clarity over extreme compactness.)

6.4 Storage and Persistence: We emphasize storing in object storage rather than a traditional database. Each analysis result is an artifact. This makes it easy to keep a history (for compliance or

future reference). It's also accessible – any engineer or tool could open the JSON and inspect or even query it (with Python scripts or by loading into a Neo4j if desired). There is no proprietary format lock-in. Moreover, using S3 means we get unlimited scalability and durability out-of-the-box. MGraph-DB's JSON serialization was explicitly meant to support “JSON and Cloud file systems as the main data store” ³⁰, which we are leveraging fully.

One can imagine these JSON graph files being stored alongside the repository in a secure bucket or even in the repo itself (though they might be large for version control systems). The persistent graph also serves as a **knowledge base** that other systems can query without rerunning the LLM stages. For instance, an IDE plugin could load the graph to provide a developer with on-the-fly insights (“This function is flagged as critical, proceed with caution”).

6.5 Querying and User Interface: With the knowledge graph in place, we then enable various ways to query and visualize it. There are multiple modes: - *Programmatic Queries*: Developers or analysts can write queries (in Cypher, Gremlin, or using MGraph's Python API) to extract information. For example, using MGraph-DB's filter interface to find all nodes of type Function with attribute `category="Data Access"` that have an outgoing edge `calls` to a node tagged `ExternalAPI`. These queries can be run as part of CI (for automated checks) or ad-hoc. - *Natural Language Queries via LLM*: We can deploy an interface (e.g., a chatbot or web UI) where users ask questions in plain English, and an LLM (with knowledge of the graph schema) translates that to graph queries, executes them, and then possibly translates results back to English. This approach was demonstrated by Zimin Chen where “the LLM generates Cypher queries based on user inquiries and uses the results to respond” ³¹. For example, the user asks, “Which functions in the payments module are not covered by unit tests?” The LLM forms a query to find Function nodes in module “payments” with no incoming edge from any Test function, runs it on the graph (via a GraphQL or Cypher endpoint), then explains the result in text. - *Visual Explorers*: We can utilize graph visualization tools (there are many that can consume JSON or connect to Neo4j-like endpoints). Security knowledge graphs can be visualized to highlight, say, an *attack path* through the code: starting from an entry point (web controller) through function calls to a database where a vulnerability lies. Graph-of-graphs structure might be visualized as collapsible groups – e.g., each module's subgraph can be collapsed into one node in a high-level view. Dinis Cruz often stresses visualization for understanding graphs, saying “without it, I can't really understand what the graphs actually look like.” ³² In our system, one could generate DOT/GraphViz diagrams or use interactive network diagrams to explore relationships. For instance, a CISO might be presented with a graph view that shows modules as nodes, with red arrows highlighting “high risk data flows” between them as identified by our analysis.

- *Alerts and Dashboards*: The stored knowledge graph can also feed dashboards. For example, we can tally vulnerabilities by category, or count functions per criticality level. Since the graph has all the info, simple scripts can roll up these numbers. A dashboard for an AppSec team might show “Total of 5 critical issues and 12 medium issues in the latest analysis, across 3 modules” and allow drilling down (the drill-down would fetch subgraph details).

6.6 Example Pipeline Execution Walk-through:

Imagine we run the pipeline on an open-source project. The source is ingested from GitHub. ASTs are built and stored. Stage 1 LLM goes through each file, outputting structured JSON. For a file `auth.py` it outputs that there's a function `login()` which calls `checkPassword()`, and it notes (from code logic or comment) that `login()` is an *authentication entry point*. We merge these into the graph. Stage 2 constructs a small context graph of security concepts (e.g., an *AuthN* node for authentication concept). Stage 3 mapping sees that `login()` likely corresponds to the *AuthN* concept, so it links the `login` function node to the *AuthN* node. It also notices (maybe by scanning code) that `login()` prints an

error to console on failure, and our vulnerability ontology says printing errors might leak info, so it links a *InformationDisclosure* issue node to `login()`. All that gets saved.

Now, stored in S3, we have a JSON that perhaps looks like:

```
{
  "nodes": [
    {
      "id": "Function_login_auth.py",
      "type": "Function",
      "name": "login",
      "module": "auth.py",
      "description": "User login endpoint",
      "tags": ["AuthEntry"]
    },
    {
      "id": "Concept_AuthN",
      "type": "SecurityConcept",
      "name": "Authentication",
      "severity": "Medium"
    },
    {
      "id": "Vuln_InfoDisclosure",
      "type": "Vulnerability",
      "name": "ErrorLeak",
      "severity": "Medium"
    }
  ],
  "edges": [
    {
      "src": "Function_login_auth.py",
      "dst": "Function_checkPassword_auth.py",
      "type": "CALLS"
    },
    {
      "src": "Function_login_auth.py",
      "dst": "Concept_AuthN",
      "type": "RELATED_TO"
    },
    {
      "src": "Function_login_auth.py",
      "dst": "Vuln_InfoDisclosure",
      "type": "HAS_VULN"
    }
  ]
}
```

(This is illustrative; actual graph JSON would have many more properties.)

From this, a query “Why is `login` considered risky?” could be answered by traversing edges from `login` function: it has a `HAS_VULN` pointing to a vulnerability node, which says an error message might leak info. And because everything is linked, we could also answer “Which parts of the system handle authentication?” by looking at all code nodes connected to the AuthN concept node.

6.7 Storage vs Database Trade-offs: Storing the graph in object storage (like S3) means query-time we either load it into memory (e.g., spin up an MGraph instance or a Neo4j instance) or use a service that can query JSON (some analytic databases can ingest JSON). This is a conscious trade-off: we prioritize write-time flexibility and simplicity over constant real-time query performance. In many AppSec and analysis scenarios, this is acceptable because the graph doesn’t need to be queried 1000 times per second; a few interactive queries or batch analyses are enough. However, if needed, one could regularly sync the JSON to a graph database to allow more concurrent queries. The architecture doesn’t forbid using a graph DB – it just decouples the analysis production (which uses MGraph and JSON) from the query serving. One could even imagine deploying the knowledge graph behind a GraphQL API for developers to query (with appropriate access control, since it might contain sensitive findings).

By not using a monolithic database, we also make the solution **more DevOps-friendly**. No special infra is needed beyond storage. The entire pipeline can run in ephemeral environments (like a GitHub Action or a Lambda). When it’s done, the output lives in S3 and can be picked up by any platform. This aligns with Cruz’s observation that “traditional graph databases did not support serverless and were... too heavy”⁹, hence a lightweight approach is better.

6.8 Integration in CI/CD: It's worth noting that this pipeline could be integrated into CI/CD pipelines for continuous analysis. For example, every pull request could trigger an update to the knowledge graph focusing on the changed parts, and if any new high-severity issue appears in the graph (e.g., a new edge connecting to a Vulnerability node), the CI could flag the PR. This moves code knowledge graphs from a one-time documentation exercise to an active part of the DevSecOps workflow. The structured data and ontologies make it feasible to automate checks that are context-aware (far beyond lint rules).

In summary, the end-to-end architecture flows like: **Code Repo → [AST Parser] → Initial Graph (syntax) → [LLM Stage 1] → Enriched Code Graph (JSON) → [LLM Stage 2 + context] → Context Graph (JSON) → [LLM Stage 3] → Linked Multi-layer Graph (JSON) → stored in S3**. Then **[Query Layer] → uses the JSON graph (via MGraph or other) to answer questions or visualize insights**.

Each arrow in that flow corresponds to a well-defined transformation, many of which are handled by LLMs cooperating with our rules. The final storage in S3 (or any object store) ensures longevity of the analysis results, while MGraph-DB provides the computation engine for assembling and querying the graph on-demand. This architecture realizes the vision of treating the codebase as a richly indexed knowledge artifact rather than just text files – analogous to how a database has an ER diagram and queryable schema, now our code has a *knowledge graph* and can be queried semantically.

7. Context-Aware Queries and Semantic Analysis Use Cases

With the semantic knowledge graph in place, we unlock a wide range of powerful analysis capabilities that are extremely valuable to CISOs, AppSec professionals, and engineering leaders. Because the graph encodes not just code structure but also context (security, business, behavior), queries can mix these concerns freely – something not possible with classical tools. Here we highlight a few high-impact use cases enabled by context-aware queries on the knowledge graph:

- **Vulnerability Graphing & Impact Analysis:** Traditional static analysis tools might tell you “function X has SQL injection.” Our graph can go further. We can query the *vulnerability layer* of the graph to pull out all nodes of type Vulnerability and then traverse to see which functions and modules they affect, and then further to see which features or business processes those modules belong to. This produces a *vulnerability graph* – essentially a subgraph of the codebase highlighting all the weaknesses and their connectivity. A CISO could ask: “Show me an attack path for this SQL injection vulnerability.” The query would start from the vulnerability node (SQLi in function F), traverse backwards through the call graph to find entry points (maybe a web API node), and output the chain of calls leading from the external input to the vulnerable query. Because the graph knows what calls what, this is straightforward path traversal. We can visualize that path to illustrate how an attacker could exploit the issue, which greatly helps in understanding severity and in discussing with developers. Moreover, by linking to business context, we can assess impact: is the vulnerable function handling sensitive data? If yes, the risk is higher. For example, a query can check if the function with SQLi is connected to a *Compliance* node (like GDPR) – if yes, that vulnerability might lead to a compliance violation if exploited. These nuanced insights help prioritize fixes and communicate to non-developers why a vulnerability matters (e.g. “this isn’t just any SQLi – it could expose user financial data, and it lies in the payment feature which is high-usage”).
- **Attack Surface Mapping:** An **attack surface** is essentially all points in the system exposed to potential attackers. With our graph, we can identify all *entry points* (e.g. public web API endpoints, UI event handlers, etc.) by querying for functions classified as externally accessible (perhaps tagged via LLM as “entry” or by naming conventions like routes). Once we have those

entry nodes, we can traverse deeper into the graph to see what they connect to. The result is a map of pathways from the “outside” to various internal components. For instance, starting from a `handleLoginRequest()` endpoint, the graph might show it calls `login()` (business logic) which calls `queryUser()` (DB access). Along that path, we might highlight if any step lacks proper security (maybe `login()` didn’t verify something). An AppSec engineer can ask: “What are all the entry points that touch the customer records database?” That query would find all paths from any node of type *Endpoint* to the node representing the *CustomerDB* or the data model of customer info. If one of those paths lacks an auth check (which might appear as no edge to an Auth concept node), that’s a potential access control gap. Another example: “List all modules reachable from the internet-facing module without crossing a security boundary.” If the architecture is supposed to have layers (say web -> service -> database, or tenant isolation), the graph can reveal if any path violates that (like a web module directly calling a DB of another context).

- **Semantic Code Search and Impact Queries:** Developers can leverage the graph for advanced code search. Because the graph is semantic, you can query not just by text but by meaning. For example: “Find all places where an admin credential is being used in code.” Instead of literally searching for a variable name, we query for nodes of type *Variable* or *Constant* that are linked to concept *Credential* or have certain patterns (the LLM might have tagged a constant as “likely API key”). Or, “Which functions perform cryptography?” – the LLM might have tagged those or we know certain API calls correspond to crypto (so those functions have edges to Crypto concept). These queries help ensure coverage: e.g., a CISO might want to review all crypto usage in the codebase for compliance; the graph provides that list in seconds, which is far more reliable than hoping developers documented everything. Another interesting query: “Show me code that changed in the last month that relates to payment processing.” If our business layer links code to domain concepts (and if we keep historical graphs or link to Git metadata), we can intersect those: find nodes under Feature=Payment that have a `last_modified` attribute within 1 month. This could focus a security review on recent high-impact changes.
- **Behavioral Reasoning and “What-If” Scenarios:** With dynamic data integrated, we could ask questions like “What happens if service X goes down? What functions are impacted?” If the graph had dependency info or call-out info (like which functions call external service X’s API), we can traverse from the node representing Service X to all code nodes depending on it. Similarly, “How would a failure in Component Y propagate?” The graph (especially if augmented with runtime call frequencies or error handling links) can show likely propagation. In a security context: “If an attacker compromises Module A, what could they reach?” This is effectively a reachability query in the graph, bounded by trust boundaries. If the modules have trust levels assigned (e.g., DMZ vs internal), the graph can simulate the attacker moving from node to node. This is taking classic threat modeling and making it a graph traversal problem – which it naturally is.
- **Policy Compliance Queries:** We can encode certain policies as graph patterns and then query for violations. For example, a policy might be “No personally identifiable info (PII) should be logged.” In graph terms, that means no edge from a PII data node to a Logging function node. We can search the graph for any data node labeled PII that has an outgoing edge to any function tagged as doing logging. If found, that’s a compliance issue. Another policy: “Encryption must be used when storing passwords.” We query: for all variables or fields named “password”, check if the function writing them has a path to a crypto routine; if not, flag it. This kind of **automated reasoning** is far more context-aware than linting because it uses the actual data flow and concept mapping, not just local patterns. Cruz’s earlier work on writing tests with ASTs for security requirements (like ensuring every web endpoint calls an auth check) is a precursor to this ³³ ³⁴. With our knowledge graph, we implement those ideas declaratively: to ensure auth

checks, we check that every endpoint function node has an edge to a Auth check node; the ones that don't are instantly known (no need to manually write a static analysis rule each time – the ontology drives it).

- **Developer Assistance and Documentation:** The graph can also be used to generate documentation or answer developer questions. A new team member could ask, “How does data validation work in this application?” The system can compile an answer by following edges: starting at input points, see where data flows and where any *Validation* concept nodes are linked. Perhaps it finds that all inputs eventually call a `sanitizeInput` function (and it can present that as: “All user input goes through `sanitizeInput` in module `Utils`, which uses OWASP ESAPI” etc.). This is essentially using the knowledge graph to automate architecture and code understanding documentation. Because our pipeline has LLMs, we could even have a step where the LLM uses the graph to produce a high-level architecture description in natural language.

7.1 Integration with Cybersecurity Workflows: The combination of graph-of-graphs and LLM means we can integrate with other security tools. For instance, if a dynamic application security testing (DAST) tool finds an endpoint vulnerable, we can map that endpoint to the code in our graph and mark those nodes. Conversely, our static graph might identify a risky area and we could direct a fuzzing tool to target it. The knowledge graph acts as a centralized model to coordinate such analysis.

An example drawn from Dinis Cruz's experiences is using graphs to model security workflows and decisions. In his OWASP Security Bot (OSBot) project, the idea was to have fact-based decision making by querying data sources ³⁵. Here, our knowledge graph is the fact repository for the code. A virtual CISO assistant could query it to answer, “Do we use any vulnerable libraries?” (graph has nodes for dependencies with version info, which we can cross-check with a vulnerability database and highlight if any match). Or, “Which parts of our system handle credit card data and who has committed to those parts?” (graph links to business concept `CreditCard` and possibly to git metadata of authorship). These are non-trivial queries that traditionally require crossing between code analysis, documentation, and people – but within the unified graph, it's just a few hops.

The **semantic richness** of the graph is what enables these advanced questions. Because we modeled technical, business, and behavioral aspects together, we avoid the classic silo where, for example, a static analysis might tell you about a vulnerability but not who to assign it to or what user data could be impacted. Our approach provides *context*, which is often called the “crown jewels” in risk management (knowing the context of an issue is as important as the issue itself). In fact, an oft-repeated mantra is “context is king” for making informed decisions, and our knowledge graph is essentially a context engine for the code.

8. Future Integration: Dynamic Data and Business Context

While our current architecture already incorporates multiple layers of context, future enhancements can push it even further by integrating more dynamic and high-level information:

- **Dynamic Runtime Data:** As mentioned, hooking in data from running systems can significantly enhance the graph. This can include **profiling data** (which functions are most frequently executed or which queries are slowest), **application logs** (link log events to the functions that emit them), **error rates** (which components often throw exceptions), and **real user monitoring data** (which features are most used by customers). By integrating these, one can prioritize issues. For example, a minor code smell in a function called a million times a day might be a higher priority than a major smell in a debug tool used once a year. The graph could have edges

that carry weights (like frequency counts) or timestamped events. One interesting dynamic aspect is **data lineage** in the application: tracking a specific piece of data as it flows through different services at runtime. That could be fed back to the graph to verify that static flow matches actual flow.

- **DevOps and Environment Context:** We can imagine linking the code graph to infrastructure or configuration. A *Deployment* node could represent a microservice deployment container, and *config* nodes could represent feature flags or environment settings. Then queries like “Is this security fix deployed to all environments?” could be answered by seeing if the code node with the fix is linked to all Deployment nodes (if we encode deployment versions). This blends into the realm of **Configuration Knowledge Graphs**, making sure the code and its config are consistent.
- **Business Process and Requirements:** In many organizations, there are separate documents for requirements, user stories, threat models, etc. These can be ingested (perhaps by LLMs) into a graph format as well. We could have *Requirement* nodes or *User Story* nodes that link to the code implementing them (maybe via mention of ticket IDs or keywords in commit messages). This would allow traceability from requirement to code to test cases (if we include testing in the graph). Similarly, *Threat Scenario* nodes from a threat modeling exercise (e.g., “attacker steals token to impersonate user”) could be linked to the code components involved in that scenario, so one can check mitigations in place for each threat.
- **Integration with MyFeeds/Cyber-Boardroom models:** Dinis Cruz’s *The Cyber Boardroom* and related efforts aim to give executives a clear picture of cyber posture. One could integrate the code knowledge graph into a larger organizational knowledge graph. For example, *Application* nodes in a company link to our code graph (maybe summarizing risk from that code). Business impact analyses (like if a certain app is compromised, what business functions are affected) could leverage our detailed graph under the hood. In essence, our code graph can feed into a *Cyber Digital Twin* of the organization. The LETS model might extend to incorporate not just LLM analysis of code, but also LLM analysis of business reports, combining both into a holistic graph. That might be further down the line, but the modular “graph of graphs” architecture sets the stage for it. Each domain (code, infrastructure, policy, people) can be a graph, and linking them yields a powerful AI-assisted governance tool.
- **Continuous Learning and Improvement:** As more data (dynamic or business) is integrated, the LLM could also be fine-tuned or few-shot-trained on our specific schema and data, improving its accuracy for future analyses. We might move from general GPT-4 to a custom model that understands “our company’s code ontology” and can answer very specific queries faster and more accurately. This could be seen as having a specialized “Codebase Copilot” that is fueled by the knowledge graph.
- **Collaboration and Feedback Loop:** We should also consider a feedback mechanism: developers or security analysts might disagree with some LLM-generated annotations or find false positives/negatives in the graph. They should be able to update the graph (mark a finding as a false positive, or add a missing link). These edits can be fed back into the next pipeline run, perhaps by adjusting prompts (“don’t flag this pattern again”) or by seeding the graph initially with the correction. Over time, this human-in-the-loop feedback makes the system smarter and more aligned with the project’s reality. Essentially, the graph becomes a living knowledge base that the team curates with the help of automation.

Integrating dynamic and business data will further fulfill the vision that “we can finally make AppSec work” effectively with GenAI assistance (as Cruz’s 2024 talks allude ³⁶). Instead of static scanners

producing PDFs that go unread, we have an interactive, queryable model of the system that stays up-to-date with both code changes and real-world usage, and ties directly into business impact. This is tremendously appealing for CISOs who constantly have to answer, “*What is our exposure in this scenario?*” or “*Are we secure against threat X?*” – questions that currently take teams days of meetings to address can be answered in minutes by querying a well-maintained semantic graph.

Incorporating future data will no doubt raise new challenges (data volume, privacy of info in the graph, etc.), but the architecture is flexible to accommodate those. In fact, the hypergraph concept mentioned earlier could come into play strongly here – treating events over time as first-class nodes (so a sequence of code deployments and incidents become part of the graph history). We foresee this approach could evolve into a form of **automated reasoning engine** where, for example, before deploying a change, one could simulate queries on the knowledge graph to see if any security rules would be violated or any past incident pattern might reoccur. Essentially, the graph and LLM become an automated reviewer or risk assessor for changes.

Conclusion

Semantic knowledge graphs for source code analysis represent a significant leap forward in how we reason about and secure complex software systems. By combining the strengths of graph data models with the interpretive power of LLMs, we turn code into **linked knowledge** that is far more accessible and actionable to both machines and humans. In this paper, we described a comprehensive architecture – co-authored through the experiences of Dinis Cruz’s pioneering work and the capabilities of ChatGPT Deep Research – that brings this vision to life.

Our approach addresses the key pain points faced by CISOs and AppSec professionals: lack of visibility, context fragmentation, and the labor-intensive nature of traditional code reviews and threat modeling. A semantic code knowledge graph provides **full visibility** of the codebase’s structure and behavior in a single pane, where each node can be explored along technical, security, and business dimensions. It contextualizes every function or module in terms of “Who uses this? Why does it matter? Is it safe?” – questions that typically require sifting through documents and tribal knowledge. With graph queries or natural language prompts, one can get instant answers, turning security and architecture analysis from a slow consultancy exercise into an interactive, continuous process.

We highlighted why a traditional relational database is ill-suited for this task – the agility and recursive complexity of the graph defies static schemas. Instead, our use of **MGraph-DB** (a memory-first, JSON-backed graph engine) gives us a nimble platform where the schema can evolve as the graph-of-graphs grows. The **Type_Safe** model ensures that even as we evolve, we don’t descend into chaos – the graph’s integrity is maintained, akin to having a strongly-typed schema without the rigidity. The LETS pipeline orchestrates LLMs to perform analysis in stages, yielding structured outputs at each step that we carefully validate and store. This achieves a rare combination of **determinism and AI-driven flexibility**: we get the creativity and insight of LLMs, but constrained within a framework that ensures outputs are usable and traceable ²² ²³ .

By grounding our arguments in Dinis Cruz’s prior projects (like MyFeeds.ai and OSBot) we demonstrated that this is not just theoretical. The use of multi-phase LLM workflows to create knowledge graphs has been proven in content personalization ³⁷ , and the creation of MGraph-DB was driven by real-world needs in modern GenAI applications ⁹ . We built on those ideas, directing them at the source code domain, which arguably stands to gain even more – because code is inherently structured data, and we’re leveraging that structure in unprecedented ways.

One of the most compelling aspects of this approach is how it can **bridge the gap between development and security, and between code and business**. A CISO or technically savvy founder can use the same knowledge graph to get answers that a developer would – but framed in business or risk terms. For instance, a founder could ask “Which user data is touched by this new feature we’re deploying?” and get an answer derived from static code facts and dynamic labels. In the past, such a question might result in an email thread and a meeting between engineering and security teams; now it could be answered via a query to the system. This has the potential to bring truly **data-driven decision-making** to software security and architecture at the executive level, fulfilling the promise of efforts like The Cyber Boardroom which aim to make cybersecurity understandable in the boardroom.

Technically, our white paper stands as a blueprint for implementing a **next-generation code analysis platform**. It underscores the need to move away from siloed tools – linters, SAST, DAST, docs – towards an integrated knowledge-centric approach. By storing the analysis results in a graph (and in simple storage), we ensure the knowledge is persistent and shareable, rather than ephemeral scan results. By using LLMs, we continually enrich this knowledge with human-like understanding, something static tools could never encode (like intent or conceptual links).

Looking ahead, as we integrate more real-time data and possibly automate the query of this graph via agents, we might achieve semi-autonomous security analysis. For example, an agent monitoring the graph could proactively query “New commit added a call to `eval()` – is this a potential issue?” and alert if yes, with context. The graph becomes not just a database but a **living security brain** for the application.

In conclusion, *Semantic Knowledge Graphs for LLM-Driven Source Code Analysis* offer a pathway to smarter, context-rich, and continuous understanding of software. By leveraging the techniques and architecture detailed in this paper, organizations can build a robust platform where code, context, and AI reasoning come together. This not only enhances security – by making it easier to spot complex vulnerabilities and attack paths – but also improves software quality and maintainability (developers get deeper insights into their code). It transforms the codebase into a **knowledge base**, one that evolves with each commit and with each threat learned.

As Dinis Cruz’s work has shown, embracing graphs and AI in application security can shift us from reactive scanning to proactive knowledge-driven defense. The methods outlined here stand on the shoulders of that work and push forward into an era where we can finally tame the complexity of modern software. With such a system, we empower teams to ask the right questions – and crucially, get the answers when they need them – about their code. This is a significant step toward software that is not only well-built but also transparently understood by its creators and protectors.

1 4 5 6 7 31 Building Knowledge Graph over a Codebase for LLM | by Zimin Chen | Medium
<https://medium.com/@ziche94/building-knowledge-graph-over-a-codebase-for-llm-245686917f96>

2 16 17 18 19 20 21 22 23 29 37 Establishing Provenance and Deterministic Behaviour in an LLM-Powered News Feed (first MyFeeds.ai MVP)
<https://www.linkedin.com/pulse/establishing-provenance-deterministic-behaviour-llm-powered-cruz-dimhe>

3 proceedings.neurips.cc
https://proceedings.neurips.cc/paper_files/paper/2024/file/3205b048f9cc54b9f7963db0b0f52d53-Paper-Datasets_and_Benchmarks_Track.pdf

8 9 10 11 12 13 14 30 **Introducing: MGraph-AI - A Memory-First Graph Database for GenAI and Serverless Apps**

<https://www.linkedin.com/pulse/introducing-mgraph-ai-memory-first-graph-database-genai-dinis-cruz-wxmde/?ref=mvp.myfeeds.ai>

15 **Type_Safe - tech docs | Dinis Cruz**

https://www.linkedin.com/posts/diniscruz_typesafe-tech-docs-activity-7286172046764462080-nluB

24 25 **OSBot Utils - Flow support | Dinis Cruz**

https://www.linkedin.com/posts/diniscruz_osbot-utils-flow-support-activity-7290696947923910656-OO7c

26 28 32 **Building Semantic Knowledge Graphs with LLMs: Inside MyFeeds.ai's Multi-Phase Architecture**

<https://www.linkedin.com/pulse/building-semantic-knowledge-graphs-llms-inside-myfeedsais-dinis-cruz-jub5e>

27 33 34 **AST (Abstract Syntax Tree). AST (Abstract Syntax Tree) is a graph... | by Dinis Cruz | Medium**

<https://medium.com/@dinis.cruz/ast-abstract-syntax-tree-538aa146c53b>

35 **GSBot Commands (Slack Bot used to access Jira data) - SlideShare**

<https://www.slideshare.net/slideshow/gsbobot-commands-slack-bot-used-to-access-jira-data/167426431>

36 **It's 2024 And, With GenAI, We Can Finally Make AppSec Work**

<https://www.youtube.com/watch?v=d61sGlgdNoo>